



Durée totale démo : 6 minutes. Ouvrir ce guide + tous les onglets *avant* d'entrer en salle. Ne pas fermer le terminal Docker.

AVANT D'ENTRER Préparation

— hors chrono

1 Démarrer Docker Desktop, puis lancer les containers

D'abord : ouvrir **Docker Desktop** depuis le Bureau ou la barre des tâches et attendre l'icône verte "Engine running".

TERMINAL `docker compose up -d --build`

Attendre ~30 sec que les 2 containers soient **healthy**, puis vérifier :

VÉRIFIER `docker compose ps`

Résultat attendu : **cesizen-app-1 Up · cesizen-db-1 Up (healthy)**

2 Ouvrir ces 6 onglets maintenant (cliquer sur chacun)

Ⓢ APP <http://localhost:8080>

Ⓢ CI/CD github.com/louisbeaujoin/cesizen/actions

Ⓢ BRANCHES github.com/louisbeaujoin/cesizen/branches

Ⓢ GIT FLOW github.com/louisbeaujoin/cesizen/network

Ⓢ ISSUES github.com/louisbeaujoin/cesizen/issues

Ⓢ KANBAN github.com/users/louisbeaujoin/projects/1

3 Checklist pré-démo

- | | |
|---|---|
| ☐ Docker Compose en marche | ☐ http://localhost:8080 répond |
| ☐ Onglet GitHub Actions ouvert | ☐ Onglet Branches ouvert |
| ☐ Onglet Issues ouvert | ☐ Terminal visible en arrière-plan |

PARTIE A Versioning + CI/CD

3 min

4 Montrer les 3 branches `github.com/louisbeaujoin/cesizen /branches`

Les 3 branches à montrer au jury :

```
main ← branche stable – tag v1.0.0 – PR uniquement
├─ develop ← intégration continue – tous les commits
│   └─ feature/historique-sessions ← fonctionnalité isolée (issue #15)
│       └─ dependabot/* ← veille automatique des dépendances
```

Puis montrer le graphe Git Flow visuel :

 `github.com/louisbeaujoin/cesizen /network` – graphe complet

Dire : « main (stable, v1.0.0) · develop (intégration) · feature/* (fonctionnalité isolée liée à un issue). »

5 Montrer le dernier run CI/CD `github.com/louisbeaujoin/cesizen /actions`

Cliquer sur le dernier run vert → montrer les 2 jobs séquentiels :

« Job "tests" → 48/48  — Job "docker" déclenché uniquement si tests passent (needs: tests). »

Si le jury demande le fichier de workflow :

 `.github/workflows/ ci.yml` – les 2 jobs séquentiels**6 Lancer les tests en direct****TERMINAL** `php artisan test`

— ou via Docker si WAMP n'est pas lancé :

DOCKER `docker compose exec app php artisan test`

Résultat attendu : **48 passed · 122 assertions · 0 failed**

PARTIE B Ticketing GitHub

2 min

7 Montrer le board Kanban `github.com/users/louisbeaujoin/ projects/1` – CESIZen Suivi

Dire : « 4 colonnes : A traiter · En cours · En test · Terminé. »

État actuel du board : #13 En cours · #14 A traiter · #15 **En cours** (feature branch) · #16 Terminé**8 Montrer les issues avec labels SLA** `github.com/louisbeaujoin/cesizen /issues`

Ouvrir issue #13 [BUG] bloquant-critique → montrer les labels SLA (HTTP 429 après 5 tentatives, throttle:5,1). Montrer la liste complète des labels :

 `github.com/louisbeaujoin/cesizen /labels` – bloquant-critique · majeur · mineur · en-cours · en-test**9 Montrer Dependabot (PRs automatiques)** `github.com/louisbeaujoin/cesizen /pulls` – filtre Dependabot (12 PRs)

Dire : « Veille hebdomadaire Composer + npm (lundi 08h00), mensuelle GitHub Actions. Labels auto : dependabot + maintenance. »

 `.github/ dependabot.yml` – configuration de la veille

PARTIE C Application Docker live

1 min

10**Ouvrir l'application****APP** `http://localhost:8080`

Se connecter avec :

LOGIN
`admin@cesizen.fr`**MOT DE PASSE**
`password`**11****Montrer les en-têtes de sécurité — F12 → Network → Headers**Appuyer sur **F12** → onglet **Network** → recharger → 1^{re} requête → **Response Headers**.Pointer : **X-Content-Type-Options** · **X-Frame-Options** · **Content-Security-Policy** · **Referrer-Policy** · **Permissions-Policy**
— ou PowerShell :**POWERSHELL**`Invoke-WebRequest http://localhost:8080 -UseBasicParsing | Select-Object -ExpandProperty Headers`

Code source du middleware :

 `app/Http/Middleware/ SecurityHeadersMiddleware.php`**12****Exercice de respiration + back-office**

1. **Exercices de respiration** → lancer → montrer l'historique personnel.
2. **Administration** → gestion des utilisateurs et contenus.

SI LE JURY DEMANDE Liens supplémentaires		— sur demande
★	Release v1.0.0 + Tags SemVer	
	 github.com/louisbeaujoin/cesizen/releases/tag/ v1.0.0	
	 github.com/louisbeaujoin/cesizen /tags	
★	PR develop → main (flux de livraison)	
	 github.com/louisbeaujoin/cesizen /pull/17 – PR v1.0.0 (mergée)	
	 github.com/louisbeaujoin/cesizen /pulls – toutes les PRs	
★	Fichiers sources clés	
	 tests/Feature/ SecurityTest.php – 4 tests sécurité automatisés	
	 docker-compose.yml – 2 services (app PHP 8.3 + MySQL 8)	
	 github.com/louisbeaujoin/ cesizen – page principale	

BONUS JURY Push en direct depuis VS Code → CI se déclenche

2 min

A Ouvrir le projet dans VS Code**TERMINAL** `code C:\wamp64\www\cesizen`

VS Code s'ouvre sur le projet. Vérifier en bas à gauche que la branche active est **develop**.



PHP · Laravel · UTF-8

B Faire une modification visible (ex : fichier README ou commentaire)

Ouvrir par exemple **README.md** et ajouter une ligne :

FICHIER README.md → ajouter : > **Démonstration soutenance Bloc 3 – Juillet 2026**

Sauvegarder avec **Ctrl+S**. Le point orange apparaît sur l'onglet Source Control.

C Ouvrir Source Control — Ctrl+Shift+G**SOURCE CONTROL****1****CHANGES**

README.md

M

+

← cliquer pour stager

Cliquer + à droite du fichier pour le stager (ou **Ctrl+Shift+P** → **Git: Stage All Changes**).

D Écrire le message de commit et committer**SOURCE CONTROL**

docs: demonstration live soutenance Bloc 3

✓

Commit

← Ctrl+Enter

Taper le message dans la zone en haut, puis **Ctrl+Enter** (ou cliquer le bouton **Commit**).

E Pusher vers GitHub**SOURCE CONTROL**1 **Sync Changes (11)**

← cliquer pour push

Le bouton **Sync Changes** apparaît en bas de la barre ou dans le panneau Source Control. Cliquer dessus = push vers **origin/develop**.

— ou depuis le terminal :

TERMINAL `git push origin develop`**F** Aller sur GitHub Actions — voir la CI se déclencher automatiquement

✓ github.com/louisbeaujoin/cesizen /actions – le run apparaît en quelques secondes

Dire : « Chaque push sur develop déclenche automatiquement la CI : job "tests" → 48/48 ✓ → job "docker". Si un test échoue, le docker build n'a jamais lieu. »

Durée attendue du run : ~2 min. Le résultat apparaît en temps réel.

⚠ PROBLÈME TECHNIQUE — COMMANDES DE SECOURS

RESTART `docker compose restart app`

REBUILD `docker compose down -v && docker compose up -d --build`

LOGS `docker compose logs -f app`

PORT `netstat -ano | findstr :8080`

En dernier recours : montrer le run CI GitHub Actions ✓ — la preuve est dans le run vert.